Listen

Learning Doc 03: Commands and the Intent Engine
How "search what is an atom" becomes an action

Detailed design, Section 2

7 October 2026

Abstract

The voice module hands us a line of text, such as `"search what is an atom"`. This document designs everything between that text and an action on your screen: the file format that describes commands, the matching algorithm that picks the right command, how default apps work, how you add and edit commands without code, and why the design is safe even when an AI writes the commands. Each new idea is introduced as a concept box.

Contents

1 The problem in one picture **2**

2 The command file **2**
2.1 Data-driven design . . . 2
2.2 Three sections: targets, defaults, commands . . . 2
2.3 Slot types . . . 3

3 Step 1: normalisation **4**

4 Step 2: matching **4**
4.1 Patterns become regular expressions . . . 4
4.2 When several patterns match: specificity . . . 4
4.3 Validation can reject a match . . . 5
4.4 Defaults fill missing slots . . . 5
4.5 No match is useful information . . . 5
4.6 The whole algorithm . . . 5

5 Finding apps by name **5**

6 Step 3: the contract with the action executor **6**
6.1 The v1 action primitives . . . 6

7 Adding and editing commands without code **6**
7.1 Where the file lives . . . 6
7.2 Validation on every load . . . 7
7.3 The command builder . . . 7
7.4 Why this is safe . . . 7

8 Growing the command list from the code side **7**
8.1 Level 1: a new command (data only, the common case) . . . 8
8.2 Level 2: a new primitive (code, the rare case) . . . 9

9 Routines: one phrase, many steps **9**

10 Undo **10**
10.1 The rules (as decided) 10
10.2 What can be reversed 10

11 Confidence thresholds **11**

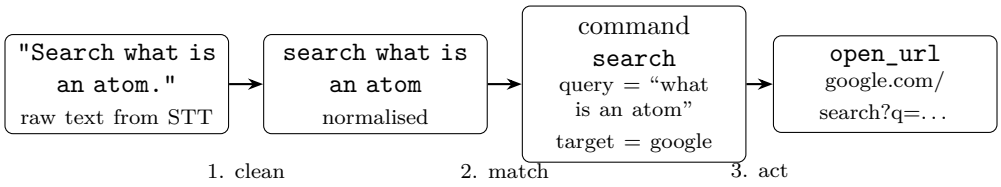
12 Commands in continuous mode **11**

13 Testing this design **11**

14 Glossary **12**

15 Check your understanding **12**

1 The problem in one picture



Three steps: **clean** the text, **match** it to a command and pull out the interesting parts, then **act**. This document covers steps 1 and 2 in detail, and the contract between step 2 and step 3.

Concept: Intent and slots

An **intent** is *what you want done* ("search"). **Slots** are the *details* needed to do it (the query "what is an atom", the target "google"). Turning a sentence into an intent plus filled slots is called **intent parsing** or **slot filling**. Siri, Alexa and Google Assistant all have this exact step.

2 The command file

2.1 Data-driven design

Concept: Data-driven design

Instead of writing behaviour as code (`if text.startswith("search")...`), you describe behaviour in a **data file**, and write one small, general piece of code that reads the file and follows it. Changing behaviour means editing data, not code.

This is what makes your requirement "add `play {query} on spotify` without touching code" possible. The file is written in **YAML**, a format designed to be easy for humans to read and edit: indentation shows nesting, `-` starts a list item, `key: value` sets a field.

2.2 Three sections: targets, defaults, commands

```
version: 1

# targets: the places Listen can send you
targets:
  google:  { search: "https://www.google.com/search?q={query}",
             home:   "https://www.google.com" }
  youtube: { play:   "https://www.youtube.com/results?search_query={query}",
             search: "https://www.youtube.com/results?search_query={query}",
             home:   "https://www.youtube.com" }
  spotify: { play:   "https://open.spotify.com/search/{query}",
             home:   "https://open.spotify.com" }

# defaults: which target a verb uses when you don't name one
defaults:
  search: google
  play:   youtube

# commands: sentence patterns -> an action primitive
commands:
  - id: search
    patterns: ["search {query} on {target}", "search {query}",
```

```
              "look up {query}", "google {query}"]
  action: open_url
  url_from: search          # use the target's "search" URL

- id: play
  patterns: ["play {query} on {target}", "play {query}"]
  action: open_url
  url_from: play

- id: open
  patterns: ["open {app}", "launch {app}", "start {app}"]
  action: launch_app

- id: close
  patterns: ["close {app}", "quit {app}", "exit {app}"]
  action: close_app

- id: volume_set
  patterns: ["volume {number}", "set volume to {number}"]
  action: set_volume
- id: volume_up
  patterns: ["volume up", "louder", "turn it up"]
  action: change_volume
  params: { step: 10 }
- id: volume_down
  patterns: ["volume down", "quieter", "turn it down"]
  action: change_volume
  params: { step: -10 }
- id: mute
  patterns: ["mute", "unmute"]
  action: toggle_mute
```

Why it is done this way

Targets are separated from **commands** because "where" and "what" change for different reasons (the single-responsibility idea from Doc 02). Look at what this buys you: `play {query} on spotify` needs *no new command at all*. The `play` command already accepts `on {target}`; you only add a `spotify` target with a `play` URL. One new line of data, and *every* verb that has a URL for Spotify starts working with it.

2.3 Slot types

Every `{name}` in a pattern is a slot, and its name decides what it may contain:

Slot	Accepts	Example
`{query}`	Any text (at least one word)	"what is an atom"
`{target}`	Only names in `targets`	"spotify"
`{app}`	An app Listen can find on your PC (Section 5)	"vs code"
`{number}`	A number, written as digits or words	"50", "fifty"

Typed slots are how the matcher tells "play *songs on the beach*" (the *query* is "songs on the beach", because "the beach" is not a target) apart from "play *lofi* on *spotify*".

3 Step 1: normalisation

Concept: Normalisation

Different-looking inputs that mean the same thing are converted into one standard form *before* comparing them. "Search what is an atom.", "search what is an atom" and "SEARCH what is an atom?" must all match the same pattern.

The cleaner, in order:

1. Lowercase everything.
2. Remove punctuation the speech engine adds (`.` `,` `?` `!`), but keep characters that matter inside queries such as `+` and `'` in "what's".
3. Remove the wake word if it leaked into the transcript ("hey jarvis search…").
4. Remove polite filler at the start and end: "please", "can you", "could you", "for me". ("Can you please search atoms for me" → "search atoms".)
5. Collapse repeated spaces.

4 Step 2: matching

4.1 Patterns become regular expressions

Concept: Regular expression (regex)

A **regex** is a tiny language for describing text patterns. For example `^search (?P<query>.+)$` means: the text *starts* (`^`) with "search ", then one or more characters (`.+`) that we capture under the name `query`, then the text *ends* (`$`). Python's built-in `re` module runs them.

You never write regexes yourself. When the command file loads, each friendly pattern is *compiled* into one:

Pattern in YAML	Regex the engine builds
`search {query}`	`^search (?P<query>.+?)$`
`play {query} on {target}`	`^play (?P<query>.+?) on (?P<target>.+?)$`

The `^` and `$` *anchors* mean the whole sentence must fit the pattern, so "research papers" does not accidentally match "search {query}".

4.2 When several patterns match: specificity

"play lofi on spotify" matches *both*

- `play {query} on {target}` with query = "lofi", target = "spotify", and
- `play {query}` with query = "lofi on spotify".

Concept: Specificity

When several rules match, prefer the **most specific** one: the one that explains more of the sentence with fixed words. Score = number of literal words in the pattern. The first pattern has 2 ("play", "on"); the second has 1. The first wins. On a tie, the command listed earlier in the file wins, so the order of the file is a tiebreaker you control.

4.3 Validation can reject a match

After a regex matches, each slot is checked against its type. If a check fails, *that candidate is thrown away* and the next-best one is tried:

```
"play songs on the beach"
  candidate 1: play {query} on {target}  target="the beach" -> not a target, REJECT
  candidate 2: play {query}              query="songs on the beach"  -> ACCEPT
```

4.4 Defaults fill missing slots

If the winning pattern has no `{target}` ("play lofi"), the engine looks up `defaults.play` → `youtube`. If the target has no URL for that verb (asking Google to "play" something), the result is a clear error message instead of a guess.

4.5 No match is useful information

If nothing matches, the engine publishes an `IntentNotMatched` event. The terminal says "Sorry, I didn't understand: …", and the sentence is logged. Later, your analytics page can list *what you tried that failed*: exactly the commands worth adding next.

4.6 The whole algorithm

```
match(text):
    text = normalise(text)
    candidates = []
    for each command, for each pattern:
        if pattern.regex fully matches text:
            slots = the captured groups
            if every slot passes its type check:
                candidates.append( (specificity, file_order, command, slots) )
    if no candidates: return NoMatch(text)
    best = highest specificity, then earliest in file
    fill missing target from defaults
    return Intent(command, slots)
```

Why it is done this way

This is a **pure function**: text in, result out, no microphone, no files, no internet, no side effects. That makes it trivial to test with a table of example sentences, and it runs in well under a millisecond, so matching costs nothing on your lightweight budget.

5 Finding apps by name

"open vs code" requires knowing that "vs code" means `C:\...\Code.exe`. Windows already keeps that knowledge: every installed app puts a **shortcut** in the Start Menu folders.

1. At startup the agent scans the Start Menu shortcut folders and builds an **app index**: display name → how to launch it. This takes a fraction of a second and is refreshed when you ask ("refresh apps") or once a day.
2. You can add **aliases** in the command file (`"code": "Visual Studio Code"`) for names the speech engine spells oddly.
3. The `{app}` slot is matched with **fuzzy matching**.

Concept: Fuzzy matching

Exact comparison fails on "vs code" vs. "Visual Studio Code" or "spotfy" vs. "Spotify". Fuzzy matching gives a **similarity score** between 0 and 1 and accepts the best candidate above a threshold (we start at 0.8 and tune it). Python's built-in `difflib` does this with no extra download.

"open google" works too: if the name is not an app but *is* a target, Listen opens that target's `home` URL.

6 Step 3: the contract with the action executor

The intent engine never performs actions itself. It publishes an `IntentMatched` event:

```
IntentMatched(
    command_id = "search",
    action     = "open_url",
    args       = { "url": "https://www.google.com/search?q=what+is+an+atom" },
    heard      = "search what is an atom",
)
```

The `actions/` module subscribes and runs it. Values placed into URLs are **URL-encoded** (spaces become `+`, `&` becomes `%26`), so a query can never break the URL or sneak in extra parameters.

6.1 The v1 action primitives

Primitive	What it does
`open_url`	Opens a web address in your default browser
`launch_app`	Starts an app from the app index
`close_app`	Asks an app's windows to close (like clicking X, so unsaved-work prompts still appear)
`set_volume`	Sets master volume to 0–100
`change_volume`	Raises or lowers volume by a step
`toggle_mute`	Mutes or unmutes
`press_keys`	Presses a keyboard shortcut (e.g. `ctrl+shift+t`) in a named app (Section 8)
`focus_app`	Brings an already-open app to the front

Watch out

Honest limitation: `play lofi` opens YouTube's *search results* for "lofi"; it does not start the first video automatically. Autoplay would need either the YouTube API or scraping the results page. It is a possible later improvement, deliberately out of v1.

7 Adding and editing commands without code

7.1 Where the file lives

- A **default** command file ships inside the agent's code (versioned in git).
- On first run it is copied to your user-data folder (`%APPDATA%\Listen\commands.yaml`). That copy is yours: edits go there, and code updates never overwrite it.
- Once sync exists (Section 5 of the design), the cloud holds the master copy, so edits made in the web app reach your laptop.

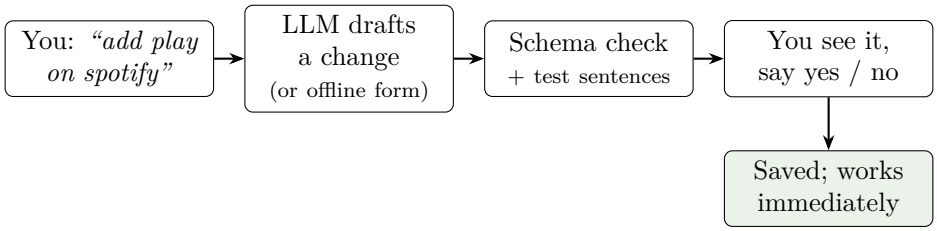
7.2 Validation on every load

Concept: Schema validation

A **schema** is a machine-checkable description of what valid data looks like: "every command must have an `id`, at least one pattern, and an `action` that is one of the registered primitives". Before using the file, the agent checks it against `shared/recipe.schema.json` (the contract from Doc 02).

An invalid entry is **skipped with a clear warning**, and every valid entry still loads. One typo never takes down the whole assistant.

7.3 The command builder



1. You describe the change in plain words: add a target, add a command, change a default, add a pattern, rename or delete something.
2. The LLM receives your request, the current file, and the schema, and returns a *proposed change* (not a rewritten file).
3. The proposal is validated against the schema and tried on a few example sentences ("play lofi on spotify" should now match).
4. You see a readable diff and approve or reject it. Nothing is saved before "yes".

Offline, a guided form asks the same questions step by step.

7.4 Why this is safe

Concept: Allowlist

An **allowlist** is a fixed list of things that are permitted; everything else is refused. Listen's registered primitives are an allowlist. A command can only ever *combine* them; there is deliberately **no** "run any program with any arguments" primitive.

Why it is done this way

An AI is writing data that your computer will act on, so you must ask: what is the worst a bad (or tricked) suggestion could do? With the allowlist, the worst case is "opens a wrong website" or "closes an app politely". Without it, the worst case is "runs any command on your PC". Combined with your approval step and URL encoding, this keeps the AI helpful but harmless. Good design limits damage *structurally*, rather than hoping nothing goes wrong.

8 Growing the command list from the code side

You will keep thinking of commands for months: "new tab", "close tab", "next song", "scroll down". Adding them must be quick *in your editor*, not only through the web app. The design gives you two levels.

8.1 Level 1: a new command (data only, the common case)

Most new commands are a keyboard shortcut or a URL in a specific app. The `press_keys` primitive covers almost all of them.

One file per area. Instead of one ever-growing file, commands live in a *folder*, one YAML file per area, and the agent loads all of them:

```
commands/
  core.yaml        # open, close, search, play, volume
  terminal.yaml    # new tab, close tab, split pane ...
  browser.yaml     # new tab, close tab, reopen tab, refresh ...
  media.yaml       # pause, next song ...
```

Context-aware commands. "New tab" means `ctrl+shift+t` in Windows Terminal but `ctrl+t` in Chrome. A command can say which app it belongs to with `when_app`:

```
# commands/terminal.yaml
- id: terminal_new_tab
  patterns: ["new tab", "open a new tab"]
  when_app: "Windows Terminal"
  action: press_keys
  params: { keys: "ctrl+shift+t" }

- id: terminal_close_tab
  patterns: ["close tab", "close this tab"]
  when_app: "Windows Terminal"
  action: press_keys
  params: { keys: "ctrl+shift+w" }

# commands/browser.yaml
- id: browser_new_tab
  patterns: ["new tab", "open a new tab"]
  when_app: ["Google Chrome", "Microsoft Edge"]
  action: press_keys
  params: { keys: "ctrl+t" }
```

Concept: Context

Context is information about the situation that changes what a sentence means. Here the context is *which app is in front* (the usage module already tracks it). A command with `when_app` is only a candidate when that app is in front, and it counts as *more specific* than a command without one, so "close tab" in Chrome closes a tab, not Chrome.

From another app. Add `focus_first: true` and a pattern like "new terminal tab": the agent brings Windows Terminal to the front, then presses the keys.

Instant reload. The agent checks the command folder's modification times every two seconds (one cheap file-system call, no extra library). Save a YAML file in PyCharm and the new command works on your next sentence, with no restart.

Two developer commands. So you never have to say a sentence out loud to test it:

```
python -m listen_agent commands check
    -> validates every file; errors point to file and line:
       terminal.yaml:14  action "press_key" is not a known primitive
```

```
                          (did you mean "press_keys"?)

python -m listen_agent commands try "close tab" --app "Google Chrome"
    -> dry run: shows the winning command, its slots and what WOULD happen,
       without doing it
```

8.2 Level 2: a new primitive (code, the rare case)

When no primitive can do the job (say, "dim the screen"), you write a new one. Adding it must not mean editing five places, so primitives use a **registry**.

Concept: Registry / plugin pattern

A **registry** is a central list that pieces of code *add themselves to*. Each primitive is one file in `actions/primitives/`, marked with a **decorator** (the `@` line, which runs when the file is imported and registers the function). At startup the agent imports every file in that folder automatically. Adding a primitive therefore means *adding one file*; no other file changes.

```
# listen_agent/actions/primitives/set_brightness.py
from listen_agent.actions.registry import primitive

@primitive('set_brightness', params={'level': int})
def set_brightness(level):
    ...  # the Windows call that sets screen brightness
```

After that, any YAML command can use `action: set_brightness`. The registry is also what `commands check` compares action names against, and the agent reports its list of primitives to the server during sync, so the AI command builder only ever proposes actions your agent actually has.

Watch out

`press_keys` makes the action allowlist weaker: some shortcuts are powerful (`win+r` opens the Run box). So `press_keys` *requires* a `when_app`, refuses a short list of system-level combinations (`win+r`, `win+x`, `ctrl+alt+del`), and has no "type arbitrary text" option. Commands the AI proposes that use `press_keys` are highlighted in the approval step.

9 Routines: one phrase, many steps

You say "DSA" and Listen opens CLion, Striver's sheet and LeetCode. That is a **routine**: a command whose action is a *list* of steps instead of a single action.

```
# commands/routines.yaml
- id: dsa
  patterns: ["dsa", "start dsa", "dsa mode"]
  steps:
    - { action: launch_app, app: "CLion" }
    - { action: open_url,   url: "https://takeuforward.org/" }
    - { action: open_url,   url: "https://leetcode.com/" }
```

Concept: Composition

Building something big out of small pieces that already work. A routine adds *no new ability*: every step is an existing primitive. That is why routines are cheap to build and safe: they inherit the allowlist automatically.

How steps run. In order, with a short pause between them so apps have time to open. If one step fails (CLion not installed), the others still run, and Listen reports "Routine DSA: 2 of 3 steps done; CLion not found". One broken step should not cost you the whole routine.

Adding a routine. Exactly like any command: add a block to `routines.yaml` in PyCharm (hot-reloaded), or describe it to the command builder ("add a routine called gym: open Spotify and play workout"), approve, done.

Later: browser profiles. Chrome can open a specific Google account's profile (`chrome.exe --profile-directory="Profile 2" <url>`). Planned for the end of the project, the design leaves room for it now: an `open_url` step will accept an optional `browser` and `profile`, mapped to a fixed list of profiles you define. Only that one flag is allowed, never arbitrary program arguments, so the allowlist stays intact.

10 Undo

10.1 The rules (as decided)

1. After an action *succeeds*, Listen remembers how to reverse it, in one **undo slot**.
2. If you say "undo" within **60 seconds**, Listen reverses it and empties the slot.
3. The slot is replaced only by the next *successfully executed* command. A misheard or unmatched sentence leaves it alone, so the undo is still there when you need it.
4. If the action has no reliable reverse, Listen says "I can't undo that" and does nothing. It never guesses.

The undo slot is separate from the history log: every action is still recorded for your analytics, whether or not it is undone.

Concept: Command pattern (undo)

Undo works by storing each action *together with its inverse*: "set volume to 80, was 40" carries the knowledge needed to restore 40. Text editors, Photoshop and games all implement undo this way. Ours holds just one entry (**single-level undo**); a list of entries would give multi-level undo.

10.2 What can be reversed

Action	Undo
`set_volume`, `change_volume`, `toggle_mute`	Restore the previous volume / mute state. Exact.
`launch_app`	Close the app instance Listen started (by its process ID, not by name).
`press_keys`	Only if the command declares one, e.g. `undo: { keys: "ctrl+w" }` for "new tab".
`close_app`	"I can't undo that": reopening loses unsaved work, so it is not a true undo.
`open_url`	"I can't undo that": the browser owns the tab, so closing it reliably is not possible.
Routines	Reverse each undoable step, newest first; report the steps that could not be undone.

11 Confidence thresholds

Two numbers decide whether Listen acts or says "didn't catch that":

- **Hearing confidence**: how sure the speech engine is about the words (Whisper reports a score per transcription).
- **Match confidence**: how well the text fits a command (1.0 for an exact pattern match, lower for fuzzy app or target names).

If either is below its threshold, Listen does nothing and says "Didn't catch that"; otherwise it takes its best guess. Each command has a `risk` level (`low` by default, `high` for closing apps), and high-risk commands need higher scores.

Concept: Calibration

Threshold values are not guessed. For the first one or two weeks, Listen logs both scores for every sentence and you mark whether it got it right. Plotting scores of right vs. wrong guesses shows where to draw the line. Choosing thresholds from measured data is called **calibration**; it is the same skill as choosing a classification cut-off in your data-analysis work.

12 Commands in continuous mode

Decision: in continuous mode, commands run *without* the wake word, from anyone in the room. The risk is **false triggers**: ordinary conversation, a video or the TV containing a sentence that matches a command. Patterns are anchored (the whole sentence must fit) and slots are typed, which already rejects most chatter ("so I closed chrome and left" does not fit `close {app}`). Free-text slots such as `play {query}`, and one-word routines such as "DSA", remain risky. Four guardrails apply *only* in continuous mode:

1. **Standalone sentences only**: a command must be surrounded by a short silence and be at most about 8 words, as a spoken command is, unlike a phrase inside a story.
2. **Stricter thresholds**: the confidence bars from Section 11 are raised.
3. **Per-command policy**: each command has `continuous: allow` (default), `confirm` (Listen asks "Close Chrome?" and waits for "yes"), or `block` (wake word required even in continuous mode). Closing apps and **all routines** default to `confirm` ("Start DSA?").
4. **Busy-room switch**: continuous mode turns off by hotkey or by saying "continuous off"; it also falls back to wake-word mode automatically when speech fills more than about 60% of the last minute (a cheap heuristic, not real speaker counting).

And if something slips through anyway, undo is the safety net.

13 Testing this design

The matcher is a pure function, so its tests are a simple table:

```
"search what is an atom"            -> search, query="what is an atom", target=google
"Can you search atoms please?"      -> search, query="atoms"
"play lofi"                         -> play,   query="lofi",  target=youtube
"play lofi on spotify"              -> play,   query="lofi",  target=spotify
"play songs on the beach"           -> play,   query="songs on the beach", target=youtube
"volume fifty"                      -> volume_set, number=50
"research papers"                   -> no match
"dsa"                               -> routine dsa (3 steps)
"undo" 30 s after "volume 80"       -> volume restored
"undo" 90 s after "volume 80"       -> "nothing to undo" (expired)
"undo" after "open google"          -> "I can't undo that"
"open vs code"                      -> open,   app="Visual Studio Code"
```

Every bug we ever find in matching becomes one more row in this table, so it can never come back unnoticed. That habit is called a **regression test**.

14 Glossary

Allowlist
A fixed list of permitted things; everything else is refused.

Anchors
Regex symbols `^` (start) and `$` (end) forcing a full match.

Data-driven design
Behaviour described in data files, followed by general code.

Fuzzy matching
Comparing text by similarity score instead of exact equality.

Intent / slot
What you want done / the details needed to do it.

Normalisation
Converting equivalent inputs to one standard form before comparing.

Pure function
Output depends only on input; no side effects. Easy to test.

Regex
A mini-language for describing text patterns.

Regression test
A test added for a fixed bug so it can never silently return.

Schema validation
Checking data against a machine-readable description of valid data.

Specificity
Preferring the rule that explains more of the input with fixed words.

URL encoding
Escaping special characters so text can be safely placed inside a URL.

YAML
A human-friendly data format using indentation for structure.

15 Check your understanding

Check yourself

1. You want "search {query} on bing". What exactly do you add to the file? Do you need a new command?
2. Why does "research papers" not match `search {query}`?
3. Walk "play songs on the beach" through the algorithm. Which candidates are produced, which is rejected, and why?
4. Why are commands in YAML rather than Python `if` statements?
5. The LLM proposes a command whose action is `run_program`. What happens, and which part of the design stops it?

6. Why is the matcher designed as a pure function? Name two benefits.
7. Your web app should show “commands you tried that failed”. Which event provides this?